Raspberry Pi Zero 2 W Wi-Fi: the CYW43436 driver, lwIP, the SD card and the two tests

Wi-Fi for the Zero 2 W under µOS++, in the shape of the Mango Pi MQ-Pro Wi-Fi tests of the RISC-V port and of the Raspberry Pi 4 B Ethernet tests: one thread owns the radio and lwIP, a second one owns the SD card, and the tests are small scripts on top.

Part	Where	What it is
SDIO host	`src/sdio.hpp`, `src/sdio.cpp`	polled SDIO host on the BCM2837's Arasan SDHCI (`0x3F300000`): pins, chip power, CMD5/3/7, CMD52, CMD53
Radio driver	`src/wlan.hpp`, `src/wlan.cpp`	CYW43436 "FullMAC" driver: firmware download, control requests, events, scan, join, Ethernet frames
Firmware	`src/fw.S`, `src/fw/`	the chip's firmware, NVRAM and regulatory data, linked into the image; origin and licence in `src/fw/README.md`
lwIP glue	`src/net.hpp`, `src/net.cpp`	interface `wl0`, DHCP client, ping, TCP echo server on port 7 (the Pi 4 B glue, on `wlan::`)

Part	Where	What it is
Bring-up and SD saver	`src/wifi_app.hpp`, `src/wifi_app.cpp`	the scripted bring-up both tests share, and the thread that owns the SD card
Board services	`src/platform.hpp`, `src/platform.cpp`	time from the ARM generic timer, delays, `kprintf`, the RAM copy of the console
SD card on SDHOST	`../../../../drivers/include/sd_sdhost.hpp`, `drivers/src/sd_sdhost.cpp`	a second `sd::SdCard` back end, for the BCM2837's other SD controller
lwIP 2.2.1	`../../../third_party/lwip/`	unmodified sources, BSD licence in `COPYING`
Tests	`../../../rpi-zero-2w/wifi-net/`, `../../../rpi-zero-2w/wifi-tcp-server/`	see below

Nothing here is compiled into the other tests: `test/rpi-zero-2w/tests.cmake` builds these sources into the INTERFACE library `aarch32-rpi-zero-2w-wifi`, which only the two Wi-Fi tests link. This is the AArch32 port's copy of the AArch64 Wi-Fi tests; the driver, the SDIO host and the lwIP glue are the same sources, with the platform services taking time and the counter from the AArch32 port (`timer_arm`, CP15) instead of the AArch64 system registers.

The hardware

Signal	Pin	Note
SDIO CLK, CMD, DAT0-3	GPIO34-39, ALT3	the Arasan SDHCI to the Wi-Fi chip; pull-ups on CMD and DAT
WL_ON	GPIO41, output	the chip's power enable, active high; pulsed low at start

Signal	Pin	Note
32.768 kHz clock	GPIO43, ALT0 (GPCLK2)	the chip's low-power clock, from the 19.2 MHz oscillator
SD card slot	GPIO48-53, ALT0	SDHOST (`0x3F202000`); the same pins in ALT3 would be the Arasan

Two SD controllers, two users. The BCM2837's Arasan controller is the only one wired to the Wi-Fi chip, and it is also the one the port's default SD card driver uses. A test that needs the radio and the card together gives the Arasan to the radio and reads the card through SDHOST, which is what the firmware and Linux do on this board. That is why this work added the `SD_BACKEND_SDHOST` back end; the other Zero 2 W tests are unchanged and still use the Arasan for the card.

The driver

The CYW43436 is a FullMAC chip: scanning, association and the WPA handshake run in its own firmware. The driver's work is to get that firmware running and to talk to it.

1. **SDIO bring-up.** Route the pins, start the 32 kHz clock, power-cycle the chip, enumerate it (CMD5, CMD3, CMD7), switch to four data lines and 25 MHz, block size 64.
2. **Backplane.** SDIO function 1 is a 32 KiB window onto the chip's internal bus. Through it the driver starts the chip's clock, reads the chip id, halts the ARM core and wakes the RAM.
3. **Firmware.** The image is written into the chip's RAM (about 440 KiB, 16 blocks per transfer), its first and last words are read back, the NVRAM text is packed and placed at the top of RAM, and the core is released. The firmware announces itself by turning on the fast clock and making function 2 ready. All of this takes about 390 ms.
4. **Which firmware.** The Zero 2 W was built with two variants of the chip; both report chip id 43430. Silicon revision 1 (the Synaptics variant) runs the `43436s` image; revision 2 and later run the `43436` image and also need a CLM (regulatory) download. This is Raspberry Pi's own mapping. Both sets are linked in and chosen at run time.
5. **Talking to the firmware.** Everything on function 2 is an SDPCM frame on one of three channels: control requests and their

responses (“ioctl” and named variables), events, and Ethernet frames. Transmission is paced by the credit the firmware returns in every frame header.
6. **Scan.** An active scan through the `escan` variable; results arrive as events. The driver reads each network’s security from its information elements (WPA, WPA2, TKIP, AES).
7. **Join.** Open, WPA-PSK or WPA2-PSK, with TKIP or AES, chosen from what the network advertised in the scan. The passphrase is handed to the firmware’s supplicant; the join is complete when the firmware reports the SSID set, the link up and the keys installed.

Polled, no interrupts. `wlan::poll()` reads what the chip holds and hands Ethernet frames to the receive handler.

Received frames are queued, never delivered from inside the driver. A frame can arrive while the driver waits inside a transmit or a control request, and that transmit may itself have been called from lwIP’s receive path. Delivering there would re-enter lwIP, which is not reentrant; the first file transfer stalled for exactly this reason. Frames now wait in a 32-entry queue until `poll()`.

Not implemented: interrupts, power save, access-point mode, WPA3, 5 GHz (the chip is 2.4 GHz only), transmit aggregation, multicast filtering.

The threads

Thread	Priority	Stack	Role
`net`	above normal	32 KiB	owns the radio and lwIP; runs the bring-up, the pings and the server; polls every 2 ms
`sd`	normal	64 KiB	owns the SD card (SDHOST) and FatFs; mounts the boot card’s FAT volume, saves a file when asked
`os_main`	normal	32 KiB	starts the `net` thread, waits

Thread	Priority	Stack	Role
			for the verdict, prints `RESULT:`

- **One thread owns the network.** The driver is polled and lwIP runs without an OS layer (`NO_SYS=1`): neither has any locking.
- **The SD card has its own thread.** A FAT write blocks for milliseconds to seconds; the `net` thread posts a request and keeps polling until it completes. The `sd` thread is started by the bring-up only after the radio is up, because both program the GPIO function registers.
- Files go to the `tests` folder of the boot card, with 8.3 names; the card is never formatted.
- The tests use the single-core start-up of `sd_test`: cores 1 to 3 stay parked.

The tests

`wifi-net`

Radio, scan, join, DHCP, ping, and a log on the SD card.

PASS when the chip answers and its firmware starts, a scan sees at least one network, the station joins the configured network, DHCP hands out an address, the gateway answers at least one of four ICMP echoes, and the run's log is written to `tests/WIFINET.LOG` and read back unchanged.

`wifi-tcp-server`

A file transfer from the host. After the same bring-up, the RFC 862 echo server on port 7: every byte a client sends comes back and is kept in RAM (up to 8 MiB). When the client closes, the stream is written to `tests/WIFIRX.BIN` on the SD card and read back, and the board prints its size and CRC-32. A log goes to `tests/WIFITCP.LOG`.

PASS when a client sent data and closed, and the stream was saved and verified. A connection that carries nothing (a port scan) is ignored. Without a client within `WIFI_TCP_WAIT_S` seconds the test fails.

The client is `wifi-tcp-server/wifi-tcp-host.py` (Python standard library only):

```
wifi-tcp-host.py 192.168.1.65 --size 262144      # a generated
        pattern
wifi-tcp-host.py 192.168.1.65 --file some.bin    # a file
```

It sends the data, checks that every byte comes back, and prints the size and CRC-32 it sent; the board's `received <n> bytes, CRC-32 <crc>` line must show the same two numbers.

Options

Given to CMake at configure time. No credentials are stored in the repository; they end up in the build folder (CMake cache and the test image) of the machine that builds.

Option	Default	Meaning
`WIFI_TEST_SSID`	empty	the network to join; without it the bring-up stops after the scan
`WIFI_TEST_PASS`	empty	its passphrase; empty means an open network
`WIFI_TEST_GATEWAY`	from DHCP	the gateway `wifi-net` pings
`WIFI_COUNTRY`	`XX`	two-letter regulatory domain (`XX`: the firmware's world-wide one)
`WIFI_TCP_WAIT_S`	`60`	seconds `wifi-tcp-server` waits for a client

The console

The full console goes to the UART as usual. The tests also keep it in RAM and send it to the host through semihosting just before the `RESULT:` line, so the runner's log has the whole run even when nobody reads the serial port. That dump is slow and makes a hardware run take about a minute.

When a bring-up fails, the `RESULT: FAIL (...)` line says where: the SDIO step, the chip id and revision, the firmware start, the join (with the firmware's last event), or DHCP.

QEMU

This port builds a `-qemu` image of every test. For these two it prints `RESULT: SKIP` and passes.

Running

From the kernel's harness, as for every rpi-zero-2w test:

```
cd "$W/micro-os-plus-iii/tests"
C=aarch32-rpi-zero-2w-cmake-gcc-debug
cmake -S . -B build/$C -D WIFI_TEST_SSID=<ssid> -D
        WIFI_TEST_PASS=<passphrase>
cmake --build build/$C --target wifi-net-hwd wifi-tcp-server-hwd
PROBE=cmsis-dap ctest --test-dir build/$C -V -R '^aarch32-rpi-
        zero-2w-wifi-net-hwd$'
PROBE=cmsis-dap ctest --test-dir build/$C -V -R '^aarch32-rpi-
        zero-2w-wifi-tcp-server-hwd$'
# while the second one waits for a client:
"$W/micro-os-plus-iii-aarch32/test/rpi-zero-2w/wifi-tcp-server/
        wifi-tcp-host.py" <address> --file <file>
```

In a scaffolded workspace the same two steps are `UOS_CMAKE_ARGS="-DWIFI_TEST_SSID=<ssid> -DWIFI_TEST_PASS=<passphrase>" ./build.sh` and `PROBE=cmsis-dap ./test.sh wifi-net --hwd-only`.

Verified

The AArch64 port's copy of these tests passed on the board on 2026-11-13 (chip 43430 revision 1, firmware 7.45.96.s1, debug configuration, CMSIS-DAP JTAG probe, a WPA/WPA2 mixed-mode home router with DHCP): `wifi-net` joined, leased, pinged the gateway and saved `tests/WIFINET.LOG`; `wifi-tcp-server` echoed and saved 256 KiB, 1 MiB and a 442211-byte file with equal CRC-32 on both sides. The AArch32 port builds both images and reports `RESULT: SKIP` for them under QEMU, but has not yet been run on the board.

Not verified: the AArch32 board run, the other chip variant (revision 2 and later, the `43436` image and its CLM download), open and WPA-only networks, a stream longer than the 8 MiB capture buffer, long runs, reconnection after the access point drops the station, and the release configuration.

Known issue: the first join attempt fails. In every run since the SD thread was added, the firmware answers the first join request with "SSID set, status 1" (refused) and accepts the second one a second later; the bring-up retries once, so the tests pass. Before the SD thread existed the first attempt succeeded. Waiting for earlier events to drain and a longer scan did not change it. The cause is not understood yet.